

HisabDo Internship Bootcamp

Software Quality Assurance (SQA) Track

Day 5 – REST API Testing Using Postman

(Test Cases Report)

Submitted By:
Eshal Kamal

Internship Duration:
60-Day Internship Bootcamp

Organization:
HisabDo

Submission Date:
5th August, 2026

Test Case 1:

Scenario: Verify that the API successfully retrieves all available products.

Attribute	Description
Test Case ID	TC-01
API/Endpoint	/products
Method	GET
Request	GET https://fakestoreapi.com/products
Expected Result	The API should return HTTP Status Code 200 OK and a non-empty JSON array containing all available products.
Actual Result	The API returned HTTP Status Code 200 OK with a non-empty JSON array containing all available products. All Postman assertions (Status code is 200, Response is an array, Response is not empty) passed successfully.
Status	Pass

Test Case 2:

Scenario: Verify that the API successfully retrieves a single product using a valid product ID.

Attribute	Description
Test Case ID	TC-02
API/Endpoint	/products/1
Method	GET
Request	GET https://fakestoreapi.com/products/1
Expected Result	The API should return HTTP Status Code 200 OK and the details of the requested product. The response should contain the required fields (title, price, category, description, and image) with valid data types.
Actual Result	The API returned HTTP Status Code 200 OK and the requested product details. All Postman assertions (Status code is 200, Product ID is correct, Required fields exist and Price is a number) passed successfully.
Status	Pass

Test Case 3:

Scenario: Verify the API response when retrieving a product using an invalid product ID.

Attribute	Description
Test Case ID	TC-03
API/Endpoint	/products/9999
Method	GET
Request	GET https://fakestoreapi.com/products/9999
Expected Result	The API should handle the invalid product ID gracefully by returning an appropriate response without exposing server errors.
Actual Result	The API returned HTTP Status Code 200 OK with an empty response body. The Postman assertions (Status code is 200 and Response body is empty) passed successfully.
Status	Pass

Test Case 4:

Scenario: Verify that the API successfully creates a new product using valid product data.

Attribute	Description
Test Case ID	TC-04
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with valid product details (title, price, description, image, and category).
Expected Result	The API should create a new product and return HTTP Status Code 201 Created with the newly created product details, including a generated product ID.
Actual Result	The API returned HTTP Status Code 201 Created with the newly created product details. All Postman assertions (Status code is 201, Product created successfully, Title is correct and Price is a number) passed successfully.
Status	Pass

Test Case 5:

Scenario: Verify that the API successfully updates an existing product using a valid product ID.

Attribute	Description
Test Case ID	TC-05
API/Endpoint	/products/1
Method	PUT
Request	PUT https://fakestoreapi.com/products/1 with valid product details (title, price, description, image, and category).
Expected Result	The API should update the specified product and return HTTP Status Code 200 OK with the updated product details.
Actual Result	The API returned HTTP Status Code 200 OK with the updated product details. All Postman assertions (Status code is 200, Product ID is correct, Title updated successfully and Price is a number) passed successfully.
Status	Pass

Test Case 6:

Scenario: Verify that the API successfully performs a partial update on an existing product using a valid product ID.

Attribute	Description
Test Case ID	TC-06
API/Endpoint	/products/1
Method	PATCH
Request	PATCH https://fakestoreapi.com/products/1 with updated title only.
Expected Result	The API should partially update the specified product and return HTTP Status Code 200 OK with the updated field.
Actual Result	The API returned HTTP Status Code 200 OK with the updated product details. All Postman assertions (Status code is 200, Product ID is correct, and Title updated successfully) passed successfully.
Status	Pass

Test Case 7:

Scenario: Verify the API response when updating a product using an invalid product ID.

Attribute	Description
Test Case ID	TC-07
API/Endpoint	/products/9999
Method	PUT
Request	PUT https://fakestoreapi.com/products/9999 with valid product details.
Expected Result	The API should reject the update request for a non-existing product ID and return an appropriate error response (e.g. 404 Not Found).
Actual Result	The API returned HTTP Status Code 200 OK and accepted the update request for product ID 9999. All Postman assertions passed, but the API allowed updating a non-existing product.
Status	Fail

Test Case 8:

Scenario: Verify the API response when partially updating a product using an invalid product ID.

Attribute	Description
Test Case ID	TC-08
API/Endpoint	/products/9999
Method	PATCH
Request	PATCH https://fakestoreapi.com/products/9999 with updated title only.
Expected Result	The API should reject the partial update request for a non-existing product ID and return an appropriate error response (e.g. 404 Not Found).
Actual Result	The API returned HTTP Status Code 200 OK and accepted the partial update request for product ID 9999. All Postman assertions passed successfully.
Status	Fail

Test Case 9:

Scenario: Verify that the API successfully deletes an existing product using a valid product ID.

Attribute	Description
Test Case ID	TC-09
API/Endpoint	/products/1
Method	DELETE
Request	DELETE https://fakestoreapi.com/products/1
Expected Result	The API should successfully delete the specified product and return HTTP Status Code 200 OK with the deleted product details.
Actual Result	The API returned HTTP Status Code 200 OK with the deleted product details. All Postman assertions (Status code is 200, Deleted product ID is correct and Response contains product title) passed successfully.
Status	Pass

Test Case 10:

Scenario: Verify the API response when attempting to delete a product using an invalid product ID.

Attribute	Description
Test Case ID	TC-10
API/Endpoint	/products/9999
Method	DELETE
Request	DELETE https://fakestoreapi.com/products/9999
Expected Result	The API should reject the delete request for a non-existing product ID and return an appropriate error response (e.g. 404 Not Found).
Actual Result	The API returned HTTP Status Code 200 OK with an empty response body. All Postman assertions (Status code is 200 and Response body is empty) passed successfully.
Status	Fail

Test Case 11:

Scenario: Verify that the API returns the specified number of products when the limit query parameter is applied.

Attribute	Description
Test Case ID	TC-11
API/Endpoint	/products?limit=5
Method	GET
Request	GET https://fakestoreapi.com/products?limit=5
Expected Result	The API should return HTTP Status Code 200 OK and exactly 5 product records in the response.
Actual Result	The API returned HTTP Status Code 200 OK with exactly 5 product records. All Postman assertions (Status code is 200, Response contains 5 products, and Response is an array) passed successfully.
Status	Pass

Test Case 12:

Scenario: Verify the API response when creating a product without the required title field.

Attribute	Description
Test Case ID	TC-12
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products without the title field.
Expected Result	The API should reject the request and return a validation error indicating that the title field is required.
Actual Result	The API returned HTTP Status Code 201 Created and created the product without the title field. All Postman assertions (Status code is 201 and Title field is missing) passed successfully.
Status	Fail

Test Case 13:

Scenario: Verify the API response when creating a product with an empty title value.

Attribute	Description
Test Case ID	TC-13
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with an empty title value.
Expected Result	The API should reject the request and return a validation error indicating that the title field cannot be empty.
Actual Result	The API returned HTTP Status Code 201 Created and created the product with an empty title value. All Postman assertions (Status code is 201 and Title is empty) passed successfully.
Status	Fail

Test Case 14:

Scenario: Verify the API response when creating a product with an invalid data type for the price field.

Attribute	Description
Test Case ID	TC-14
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with price set to "abc" instead of a numeric value.
Expected Result	The API should reject the request and return a validation error indicating that the price field must be a numeric value.
Actual Result	The API returned HTTP Status Code 201 Created and created the product with price as "abc".
Status	Fail

Test Case 15:

Scenario: Verify the API response when an invalid endpoint is requested.

Attribute	Description
Test Case ID	TC-15
API/Endpoint	/product
Method	GET
Request	GET https://fakestoreapi.com/product using an incorrect endpoint name.
Expected Result	The API should return HTTP Status Code 404 Not Found indicating that the requested endpoint does not exist.
Actual Result	The API returned HTTP Status Code 404 Not Found for the incorrect endpoint.
Status	Pass

Test Case 16:

Scenario: Verify the API response when creating a product with duplicate product data.

Attribute	Description
Test Case ID	TC-16
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with product data identical to an existing product.
Expected Result	The API should prevent duplicate product creation and return an appropriate validation response if duplicate data is not allowed.
Actual Result	The API returned HTTP Status Code 201 Created and created a new product with duplicate product details.
Status	Fail

Test Case 17:

Scenario: Verify that the product response contains all required fields.

Attribute	Description
Test Case ID	TC-17
API/Endpoint	/products/1
Method	GET
Request	GET https://fakestoreapi.com/products/1
Expected Result	The API should return HTTP Status Code 200 OK and the response should contain all required fields including id, title, price, category, image, and rating.
Actual Result	The API returned HTTP Status Code 200 OK with all required fields present. All Postman assertions (Status code is 200, Required fields exist and Response contains rating object) passed successfully.
Status	Pass

Test Case 18:

Scenario: Verify that product response fields contain the correct data types.

Attribute	Description
Test Case ID	TC-18
API/Endpoint	/products/1
Method	GET
Request	GET https://fakestoreapi.com/products/1
Expected Result	The API should return HTTP Status Code 200 OK and response fields should have correct data types: id as number, title as string, price as number, and category as string.
Actual Result	The API returned HTTP Status Code 200 OK with correct data types. All Postman assertions (Status code is 200, ID is a number, Title is a string, Price is a number, and Category is a string) passed successfully.
Status	Pass

Test Case 19:

Scenario: Verify that the API response time is within an acceptable limit.

Attribute	Description
Test Case ID	TC-19
API/Endpoint	/products/1
Method	GET
Request	GET https://fakestoreapi.com/products/1
Expected Result	The API should return HTTP Status Code 200 OK and the response time should be less than 1000ms.
Actual Result	The API returned HTTP Status Code 200 OK and the response time was less than 1000ms. All Postman assertions (Status code is 200 and Response time is less than 1000ms) passed successfully.
Status	Pass

Test Case 20:

Scenario: Verify the API response when creating a product with the minimum boundary value for price.

Attribute	Description
Test Case ID	TC-20
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with price value set to 0.
Expected Result	The API should reject a product with zero price and return a validation error because product price should be greater than zero.
Actual Result	The API returned HTTP Status Code 201 Created and created the product with price: 0.
Status	Fail

Test Case 21:

Scenario: Verify the API response when creating a product with a negative price value.

Attribute	Description
Test Case ID	TC-21
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with price value set to -10.
Expected Result	The API should reject the request and return a validation error because product price cannot be negative.
Actual Result	The API returned HTTP Status Code 201 Created and created the product with a negative price value (-10).
Status	Fail

Test Case 22:

Scenario: Verify the API response when creating a product with a very large price value.

Attribute	Description
Test Case ID	TC-22
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with price value set to 999999999.
Expected Result	The API should validate the maximum allowed price range and reject values exceeding the defined limit.
Actual Result	The API returned HTTP Status Code 201 Created and created the product with a very large price value (999999999).
Status	Fail

Test Case 23:

Scenario: Verify the API response when creating a product with an extremely long title value.

Attribute	Description
Test Case ID	TC-23
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products with a very long value in the title field.
Expected Result	The API should validate the maximum allowed title length and return a validation error if the title exceeds the limit.
Actual Result	The API returned HTTP Status Code 201 Created and created the product with an excessively long title.
Status	Fail

Test Case 24:

Scenario: Verify the API response when creating a product without the required price field.

Attribute	Description
Test Case ID	TC-24
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products without the price field.
Expected Result	The API should reject the request and return a validation error indicating that the price field is required.
Actual Result	The API returned HTTP Status Code 201 Created and created the product without the price field.
Status	Fail

Test Case 25:

Scenario: Verify that the API provides an appropriate error response when requesting a product with an invalid ID format.

Attribute	Description
Test Case ID	TC-25
API/Endpoint	/products/abc
Method	GET
Request	GET https://fakestoreapi.com/products/abc using a non-numeric product ID.
Expected Result	The API should return an appropriate error response (such as 400 Bad Request or 404 Not Found) with an error message indicating an invalid product ID.
Actual Result	The API returned HTTP Status Code 200 OK with an empty response body and no error message.
Status	Fail

Test Case 26:

Scenario: Verify that the API response contains required HTTP headers.

Attribute	Description
Test Case ID	TC-26
API/Endpoint	/products/1
Method	GET
Request	GET https://fakestoreapi.com/products/1
Expected Result	The API should return HTTP Status Code 200 OK and the response should contain the required Content-Type header.
Actual Result	The API returned HTTP Status Code 200 OK and the Content-Type header was present. All Postman assertions (Status code is 200 and Content-Type header exists) passed successfully.
Status	Pass

Test Case 27:

Scenario: Verify API behavior when the Content-Type header is missing during product creation.

Attribute	Description
Test Case ID	TC-27
API/Endpoint	/products
Method	POST
Request	POST https://fakestoreapi.com/products without Content-Type: application/json header.
Expected Result	The API should either reject the request with an appropriate validation error or process it consistently with the correct response format and status code.
Actual Result	The API returned HTTP Status Code 200 OK with only the generated product ID in the response body ({"id":21}).
Status	Fail

Test Case 28:

Scenario: Verify that the API successfully retrieves all available product categories.

Attribute	Description
Test Case ID	TC-28
API/Endpoint	/products/categories
Method	GET
Request	GET https://fakestoreapi.com/products/categories
Expected Result	The API should return HTTP Status Code 200 OK and a non-empty JSON array containing all available product categories.
Actual Result	The API returned HTTP Status Code 200 OK with a non-empty JSON array containing the categories: electronics, jewelery, men's clothing and women's clothing. All Postman assertions (Status code is 200, Response is an array and Response is not empty) passed successfully.
Status	Pass

Test Case 29:

Scenario: Verify that the API successfully retrieves products belonging to a specific category.

Attribute	Description
Test Case ID	TC-29
API/Endpoint	/products/category/electronics
Method	GET
Request	GET https://fakestoreapi.com/products/category/electronics
Expected Result	The API should return HTTP Status Code 200 OK and a non-empty JSON array containing only products from the electronics category.
Actual Result	The API returned HTTP Status Code 200 OK with a non-empty JSON array containing products from the electronics category. All Postman assertions (Status code is 200, Response is an array and All products belong to electronics category) passed successfully.
Status	Pass

Test Case 30:

Scenario: Verify that the API successfully retrieves products sorted in descending order.

Attribute	Description
Test Case ID	TC-30
API/Endpoint	/products?sort=desc
Method	GET
Request	GET https://fakestoreapi.com/products?sort=desc
Expected Result	The API should return HTTP Status Code 200 OK and a non-empty JSON array containing products sorted in descending order.
Actual Result	The API returned HTTP Status Code 200 OK with a non-empty JSON array of products. All Postman assertions (Status code is 200, Response is an array and Response is not empty) passed successfully.
Status	Pass